

別案を作った理由について

- スマホ表示でサムネイルをつけた
- **large: true** をデータに持たせることを見送った
- image要素をラッパーで包みグローバルから呼び出す方法をとった
- グローバルの要素は、必要であればmodule.scssでスタイルを変更できるようにした
- テンプレートリテラルと **.trim()** で、自然な段落入力ができる形にした
- コンテンツ幅の指定の方法を **.inner** と **content-width** として **mixin** に整理した
- **clamp()** を毎回計算を求めるスクリプトを作った

元の構造に変更を加えました。

```
<article class="drama open-modal" data-modal="modal-stranger-things">
  <div class="flex">
    <div class="thumbnail">
      
    </div>
    <h4>Stranger Things</h4>
    <p class="score">9/10</p>
  </div>
  <p class="information">
    5 seasons / SF • Juvenile<br />
    2016 / NETFLIX
  </p>
</article>
```

```
article.drama
├── div.flex
│   ├── div.thumbnail
│   ├── h4
│   └── p.score
└── p.information
```

気になったのは、主に次の点です。

- **.flex**が「何のまとまりか」を表していない
- タイトルとスコアは同じ箱なのに、作品情報だけ外にある
- 作品番号をCSSの機能を使っていたのを構造化してみた
- **.score**をカード全体基準で配置しにくい
- **h4**を**h3**に変更した

ただし、元のHTMLが文法的に間違っているわけではありません。各要素の役割を構造から読み取りやすくしてみるという試みです。

```
<article className={styles.card}>
  <div className="img-wrapper">
    <Image
      src={drama.image}
      alt={` ${drama.title}のサムネール`}
      width={1920}
      height={1076}
    />
  </div>

  <p className={styles.score}>
    {drama.score}/10
  </p>

  <div className={styles.body}>
    <div className={styles.heading}>
      <span className={styles.order}>
        {displayOrder}
      </span>

      <h3 className={styles.title}>
        {drama.title}
      </h3>
    </div>

    <p className={styles.information}>
      {drama.seasons} seasons /{" "}
      {drama.genres.join(" • ")}
      <br />
      {drama.year} / {drama.platform}
    </p>
  </div>
</article>
```

構造はこうなりました。

```
article.card
├── div.img-wrapper
│   └── Image
├── p.score
├── div.body
│   ├── div.heading
│   │   ├── span.order
│   │   └── h3.title
│   └── p.information
```

役割は、

```
img-wrapper  
→ 作品画像  
  
score  
→ カード全体に対する点数  
  
body  
→ 文字情報全体  
  
heading  
→ 番号とタイトル  
  
information  
→ シーズン、ジャンル、年、配信元
```

さらにスマホでは、同じHTMLのまま、

画像 | 本文 | スコア

と配置できます。

PCでは、

画像
番号・タイトル
作品情報

に戻せます。

まず最初に、この別案は、元のコードを否定して置き換えるために作ったものではありません。

教材を自分で進めながら、元のHTMLをNext.jsへ移していく過程で、

「このデータは何のために持つのか」

```
featured: true,  
large: false,  
publishedAt: "2026-09-17T16:05:20+09:00",  
year: 2016,
```

「この表示はデータで決めるべきか、CSSで決めるべきか」

「チームで更新していく場合、どこに責任を持たせると分かりやすいか」

という疑問がいくつか出てきました。

そこで、元の設計をできるだけ残しながら、別の考え方でも一度実装して比較できるようにしました。

完成案というより、設計を話し合うための試作です。

1. データと表示の役割を整理しました

作品データには、作品そのものに関する情報を持たせました。

```
title
image
score
genres
year
platform
featured
publishedAt
```

一方で、カードを大きく表示するか、小さく表示するかという情報は、作品そのものの属性ではなく、一覧のレイアウトに関する情報だと考えました。

そのため、今回は `large: true` のような指定をデータから外し、一覧の先頭にあるカードをCSSで大きく表示する構造にしています。

```
featured
→ 特集する作品なのでデータで指定する

publishedAt
→ レビューの公開日時なのでデータで管理する

先頭カードを大きくする
→ 一覧上の見せ方なのでCSSで指定する
```

このように、内容と見た目の責任を分けました。

2. レビューサイトとして公開日時を持たせました

元のデータには作品の公開年である `year` があります。

しかし、レビューサイトとして一覧を並べる場合は、作品が公開された年とは別に、そのレビューを掲載した日時が必要だと考えました。

そこで、次の値を追加しました。

```
publishedAt: "2026-09-17T16:05:20+09:00"
```

通常の一覧は、`publishedAt` を基準に新しいレビューから順番に並べます。

```
const otherDramas = dramas
  .filter((drama) => !drama.featured)
  .sort(
    (a, b) =>
      Date.parse(b.publishedAt) -
      Date.parse(a.publishedAt)
  );
```

これによって、配列内でデータを手作業で並べ替えなくても、新しく公開したレビューが先頭へ来ます。

さらに、その先頭カードをCSSで大きくすれば、

新しく公開したレビュー
↓
一覧の先頭に表示される
↓
先頭なので大きく表示される

という一貫した仕組みになります。

特に強く見せたい作品については、`featured: true`としてHeroへ表示します。

つまり、

Featured
→ 編集側が意図的に選ぶ特集作品

一覧の大きなカード
→ 最新のレビュー

という役割分担です。

3. コンポーネントの責任を分けました

今回は、次のように分けました。

```
DramaArchive
├── FeaturedDrama
├── DramaList
│   └── DramaCard
```

`DramaArchive`は、データを受け取り、特集作品と通常作品に分けます。

`FeaturedDrama`は、特集作品の表示を担当します。

DramaListは、配列を`map()`で展開し、一覧として並べます。

DramaCardは、作品一件分の表示を担当します。

これによって、データの選択、一覧の生成、一件分の表示を、それぞれ別の場所で考えられるようにしています。

4. スマートフォンでもサムネイルを残しました

元のスマートフォン表示では、一覧のサムネイルが非表示になっていました。

ただ、今回は作品レビューの一覧なので、画像も作品を識別する重要な情報だと考えました。

そのため、スマートフォンではPC版のカードをそのまま縮小するのではなく、

縦長のサムネイル
作品番号・タイトル・作品情報
黄色いスコア

という横並びのカードへ変更しました。

PCでは画像を大きく見せ、スマートフォンでは一覧性を維持しながら画像も残す設計です。

JSX側で画面幅を判断する処理は入れていません。同じHTML構造のまま、CSSのメディアクエリでレイアウトだけを変更しています。

5. CSSの責任範囲も整理しました

サイト全体で共通するものは、グローバルスタイルに置きます。

body
見出しの基本設定
共通の画像ラッパー
ページ全体の幅

各コンポーネントだけで使うものは、対応するCSS Moduleに置きます。

DramaCardの点数・本文・タイトル
FeaturedDramaの引用・タイトル・点数
DramaListのグリッド配置

例えば、共通の画像ラッパーはグローバルに置いています。

```
.img-wrapper {  
  aspect-ratio: 16 / 9;  
  overflow: hidden;
```

```
& > img {  
  display: block;  
  width: 100%;  
  height: 100%;  
  object-fit: cover;  
}  
}
```

一覧の先頭カードだけ比率を変更するときは、CSS Moduleからグローバルクラスを指定します。

```
.item:first-child {  
  :global(.img-wrapper) {  
    aspect-ratio: 4 / 3;  
  }  
}
```

これにより、JSX側で条件によってクラス名を組み立てなくても、レイアウトに関する処理をSCSS側で完結できます。

6. 作業中に二つの補助ツールができました

今回の作業では、実装そのものとは別に、今後も利用できる二つの補助ツールができました。

clamp()計算スクリプト

最小寸法、最大寸法、開始画面幅、終了画面幅を入力すると、CSSの`clamp()`を計算するスクリプトです。

```
npm run clamp
```

例えば、次のような値を生成できます。

```
clamp(15px, calc(4.078px + 2.913vw), 60px)
```

毎回手作業で計算する必要がなく、チーム内でも同じ計算方法を共有できます。

content-width mixin

ページ内のコンテンツ幅を統一するためのmixinです。

```
@mixin content-width(  
  $max-width: 900px,  
  $gutter: 15px  
) {  
  width: 100%;
```

```
max-width: $max-width;  
margin-inline: auto;  
padding-inline: $gutter;  
}
```

基本形は、次のように使います。

```
.inner {  
  @include content-width;  
}
```

必要な場合だけ、画面幅に応じて余白や最大幅を変更できます。

```
.inner {  
  @include content-width;  
  
  @include mq(md) {  
    padding-inline: 30px;  
  }  
  
  @include mq(lg) {  
    max-width: 1200px;  
    padding-inline: 60px;  
  }  
}
```

これも、単に記述を短くするためというより、サイト全体でコンテンツ幅の考え方を統一するためのものです。

7. 今回相談したいこと

今回の別案で相談したいのは、見た目の好みだけではありません。

主に、次の設計判断が妥当か確認したいと思っています。

1. レビューの公開日時をデータとして持つこと
2. 公開日時順に一覧を生成すること
3. Featuredと最新レビューの役割を分けること
4. largeのようなレイアウト情報をデータから外すこと
5. スマートフォンでもサムネイルを表示すること
6. グローバルSCSSとCSS Modulesの責任を分けること

元の案を一方的に変更したいのではなく、実際にもう一つの形を作ったうえで、どちらが今後の運用やチーム制作に適しているかを話し合いたい、という意図です。

教材としても、

配列
map()
props
データの絞り込み
並べ替え
コンポーネントの責任分離
CSS Modules
レスポンシブ対応

が一つの流れとしてつながるため、学習内容との相性もよいと考えています。